

Start testing

1. Course Content

Note: This program runs on a sandbox map; you will need to adjust the program for your personal map. This course is only for debugging the visualization of car lane line recognition

Basic: Start the autonomous driving program and observe lane line recognition results.

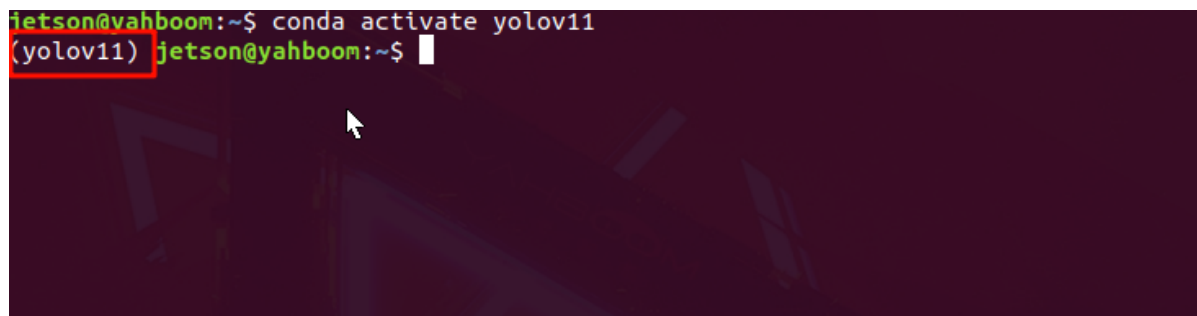
Advanced: Master lane line detection principles combined with debug mode.

2. Pre-test Preparation

- Enter conda virtual environment in advance

Because the Jetson Nano board's native Python environment only has 3.6 and 2.7, which cannot install YOLOv11, all autonomous driving environments here are built in a virtual environment. Enter the virtual environment by typing the following command in the terminal:

```
conda activate yolov11
```



- Adjust test parameters

Parameter path:

```
/home/jetson/yahboomcar_auto/src/auto_driver/config/auto_drive_node.yaml
```

```
turn_radius: 0.4
linear_speed: 0.13
angular_speed: -0.55
duration: 2.0
response_distance_1: 2900
response_distance_2: 132
cooldown_time: 0.2
DEBUG_MODE: True
DEBUG_MODE_2: False
START_AutoDrive: False
Kp: -0.62
Ki: 0.0
Kd: -0.1
max_integral: 1000.0
image_size: [640, 480]
```

Note: DEBUG_MODE and DEBUG_MODE_2 cannot be enabled simultaneously

DEBUG_MODE: Change to True, True will display the lane line interface and print debug logs.

DEBUG_MODE_2: Change to True, True will display the road sign recognition interface and print debug logs.

(For actual running, it is recommended to close these two debug modes because the program uses `self.get_logger().info()` to print logs, which will occupy ROS resources and cause frame drops, leading to poor autonomous driving performance)

Change START_AutoDrive to False, the main switch for car movement. Change to True to make the car move.

After modifying parameters, recompile, and the next run will use the modified parameters.

```
cd ~/yahboomcar_auto/  
catkin_make
```

- Camera position

Camera placement positions may vary. The current program runs with the depth camera angle shown in the figure below for reference



3. Running Example Program

Autonomous driving source code path:

```
/home/jetson/yahboomcar_auto/src/auto_driver/scripts/auto_drive.py
```

- Start camera + chassis + autonomous driving node (need to run in virtual environment terminal)

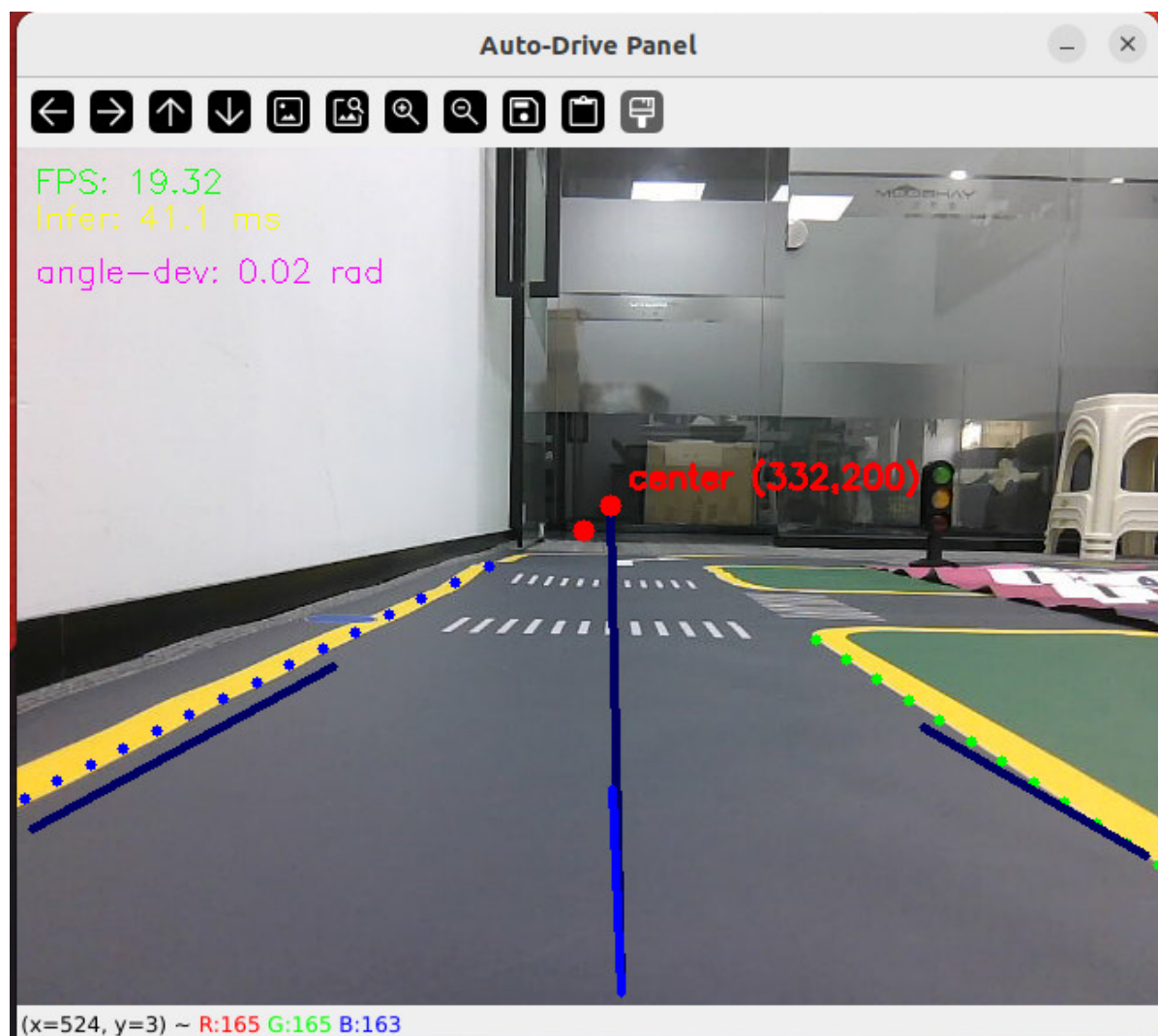
```
roslaunch auto_driver auto_drive.launch
```

```
(yolov11) jetson@yahboom:~$ roslaunch auto_driver auto_driver.launch
... logging to /home/jetson/.ros/log/2add7f36-d0c0-11f0-9d82-48b02d5ab21e/roslaunch-yahboom-15809.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://yahboom:44059/

SUMMARY
=====
PARAMETERS
* /ascamera_hp60c/color_pcl: False
* /ascamera_hp60c/confiPath: /home/jetson/yahb...
* /ascamera_hp60c/depth_height: 480
* /ascamera_hp60c/depth_width: 640
* /ascamera_hp60c/fps: 30
* /ascamera_hp60c/pub_mono8: False
* /ascamera_hp60c/pub_tfTree: True
* /ascamera_hp60c/rgb_height: 480
* /ascamera_hp60c/rgb_width: 640
* /ascamera_hp60c/usb_bus_no: -1
* /ascamera_hp60c/usb_path: null
* /auto_driver/DEBUG_MODE: False
* /auto_driver/DEBUG_MODE_2: True
* /auto_driver/Kd: -0.1
* /auto_driver/Ki: 0.0
* /auto_driver/Kp: -0.62
* /auto_driver/START_AutoDrive: False
* /auto_driver/angular_speed: -0.55
* /auto_driver/cooldown_time: 0.2
* /auto_driver/duration: 2.0
* /auto_driver/image_size: [640, 480]
* /auto_driver/linear_speed: 0.13
* /auto_driver/max_integral: 1000.0
* /auto_driver/response_distance_1: 2900
* /auto_driver/response_distance_2: 132
* /auto_driver/turn_radius: 0.4
* /auto_driver/turn_speed: 0.4
```

After successful startup, the display will show but the car will not move, mainly for testing lane line recognition. (If you want the car to move, change the parameter START_AutoDrive in auto_drive_node.yaml to True)



If you can see the screen shown in the figure above, it means the model and program are working normally. The blue dots indicate that this side is the left side of the car, and the yellow dots indicate the right side of the car. The blue in the middle is the angle between the center axis and the extension line. Ensure that the yellow lines on the road can all be recognized before

proceeding to the next debugging step.

4. Lane Line Detection Principle and Source Code Analysis

4.1 Lane Line Detection Principle

1. Lane line detection first uses the YOLOv11 object detection model to detect the distribution area of lane lines

[!NOTE]

- If you need to train a custom lane line detection model, see YOLOv11 model training

2. Then use Hough line detection to find the longest line segment in the detection box. By determining the line slope, left and right lane lines are classified

[!TIP]

- Lane line detection can also be implemented through YOLOv11's segmentation model and OBB rotation detection model, or other types of recognition models, but on Jetson devices, object detection models have faster inference speeds and better real-time performance in actual testing.
- YOLOv11 object detection model for lane line recognition can only detect the distribution of lane lines but cannot directly distinguish between left and right lane lines, because left and right lane line features are extremely similar. If data is annotated as left and right lane lines during the data annotation stage, the probability of misjudgment between left and right lane lines during detection will be very high. Therefore, this method uses YOLOv11 + OpenCV Hough detection combination for comprehensive determination of left and right lane lines.

4.3 Lane Line Detection Source Code

- YOLOv11 inference source code:

```
/home/jetson/yahboomcar_auto/src/auto_driver/scripts/utils/yolov11_infer.py
```

- auto_drive.py main program source code:

```
/home/jetson/yahboomcar_auto/src/auto_driver/scripts/auto_drive.py
```

4.3.1 YOLOv11 Model Inference Source Code

- In yolov11_infer.py, YOLO_MODEL is the base class for YOLO detection, used to extend different YOLO recognition model programs

```

class YOLO_MODEL(ABC):
    '''Model base class'''
    def __init__(self):
        self.yolo_model=None
    @abstractmethod
    def get_infer_result(self,source):
        """Return the model recognition result"""
        pass
    @abstractmethod
    def init_yolo_engine(self):
        """Load model"""
        pass

```

- Lane_Detect is a subclass inheriting from YOLO_MODEL, containing various methods for lane line detection

```

class Lane_Detect(YOLO_MODEL):
    """Lane line detection model"""
    def __init__(self,config_file_path:str,
                  midline_file_path:str):
        with open(config_file_path, "r") as file:
            full_config = yaml.safe_load(file)
            self.config_param = full_config.get("line_detection", {})
        with open(midline_file_path, "r") as file:
            self.point_config = yaml.safe_load(file)
    -----

```

- get_infer_result is the method for inferring images to get lane line detection results

```

def get_infer_result(self,source,stream:bool=True):
    '''Obtain the reasoning result of a single frame image'''
    result =
self.yolo_model(source,stream=stream,conf=self.conf,iou=self.iou,rect=self.rect,
half=self.half,

device=self.device,batch=self.batch,max_det=self.max_det,augment=self.augment,
verbose=self.verbose,classes=self.classes)

    return result

```

- __draw_dotted_line method is used to draw the detected detection boxes as visualized lane lines (dotted line segments)

```

def __draw_dotted_line(self, image, start_point, end_point, color=(255, 0, 0),
thickness=3, dot_spacing=20):
    """Draw dotted line segments composed of points"

    # Calculate the distance and direction between two points
    dx = end_point[0] - start_point[0]
    dy = end_point[1] - start_point[1]
    distance = np.sqrt(dx**2 + dy**2)

    num_dots = int(distance / dot_spacing)
    points = []
    for i in range(num_dots + 1):
        t = i / num_dots if num_dots > 0 else 0

```

```

        x = int(start_point[0] + t * dx)
        y = int(start_point[1] + t * dy)
        points.append((x, y))
    for point in points:
        cv2.circle(image, point, thickness, color, -1)
    return image

```

4.3.2 Main Program for Lane Line Detection Processing

- `image_callback` is the callback function for subscribing to camera topics, used to receive real-time camera images, then convert them to OpenCV format and save them as global `self.cv_image`.

```

def image_callback(self, msg):
    '''Image callback function'''
    try:
        self.cv_image = self._imgmsg_to_cv2(msg)
    except CvBridgeError as e:
        rospy.logerr("CvBridge error: {}".format(e))
    return

```

- `handle_lane` method is responsible for taking image frames from the `Lane_Detect` class's cache queue for processing. The processing flow is: 1. Global image frame → 2. Model inference to get lane line detection results → 3. Classify left and right lane candidate boxes → 4. Filter optimal left and right lane boxes → 5. Visualize results → 6. Loop waiting for next frame

The program for classifying left and right lane lines is as follows:

```

def _process_lane_detection(self, results):
    '''Process lane detection results'''
    left_candidates = []
    right_candidates = []

    for res in results:
        boxes = res.boxes
        if boxes is not None:
            for i in range(len(boxes)):
                box_coords = boxes.xyxy[i].tolist()
                if box_coords[0] < 320: # Left half of image
                    left_candidates.append(box_coords)
                else: # Right half of image
                    right_candidates.append(box_coords)

    left_box = max(left_candidates, key=lambda box: box[3]) if left_candidates else None
    right_box = max(right_candidates, key=lambda box: ((box[2] - box[0])**2 + (box[3] - box[1])**2)**0.5) if right_candidates else None

    return left_box, right_box

def _handle_lane_detect_result(self, left_box, right_box, frame):
    '''Process lane detection results and control vehicle'''
    error_angular = [0.0, 0.0]
    if self.flag != False:
        return frame

```

```

        # Classify processing based on detected lane line conditions
        if left_box is not None and right_box is not None and left_box[0] <
right_box[0]:
            # Both sides have lane lines
            self.error_mode = 0
            frame = self.lane_detect.draw_left_line(frame, left_box)
            frame = self.lane_detect.draw_right_line(frame, right_box)
            frame, x_center, y_center =
self.lane_detect.draw_center_point(frame, left_box, right_box)
            error_angular = self.lane_detect.cal_error_angular(x_center,
y_center)

            # PID control
            if abs(error_angular[1]) >= 0.03 and abs(error_angular[1]) < 0.3:
                angular_z = self.pid_controller.pid_control(error_angular[1])
                angular_z = max(min(angular_z, 0.3), -0.3)
                self._set_current_speed(linear_x=self.drive_speed,
angular_z=angular_z)
            else:
                self._set_current_speed(linear_x=self.drive_speed,
angular_z=0.0)

        elif left_box is not None and right_box is None and left_box[0] < 320:
            # Only left lane line
            self.error_mode = 1
            self.lane_detect.prev_center = None
            frame = self.lane_detect.draw_left_line(frame, left_box)

            if_infer = self.lane_detect.IF_lane_intersection(left_box)
            if if_infer is not None:
                self._set_current_speed(linear_x=self.drive_speed,
angular_z=self.angular_speed)
            else:
                self._set_current_speed(linear_x=self.drive_speed,
angular_z=0.0)

        elif right_box is not None and left_box is None:
            # Only right lane line
            self.error_mode = 2
            self.lane_detect.prev_center = None
            frame = self.lane_detect.draw_right_line(frame, right_box)
            self._set_current_speed(linear_x=self.drive_speed, angular_z=0.0)
        else:
            # No lane lines detected
            self.error_mode = 99
            self.lane_detect.prev_center = None
            self._set_current_speed(linear_x=self.drive_speed, angular_z=0.0)

        # Display angle deviation
        cv2.putText(frame, "Angle Dev: {:.2f} rad".format(error_angular[1]),
(10, 105),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 0, 255), 2)
        frame = self.lane_detect.draw_midline(frame)

        return frame

```

